

Omni-AI Mark II

Complete Codebase Audit & Feature Report

Audit date: 2026-05-25 | Method: Code verification only (not README/roadmaps)
Alembic head: 20260608_0016 | Tests: 256 passed | Backend coverage: 71% (80% gate fails)

1. Executive Summary

Omni-AI Mark II is a self-hosted AI workspace web application. Users sign in with Google or GitHub, upload documents or sync a GitHub repository, and chat with an LLM that answers using retrieval-augmented generation (RAG) with citations.
Stack: Next.js frontend, FastAPI backend, PostgreSQL, ChromaDB, optional Redis/RQ, Groq (default) for LLM inference.

What it is NOT

- Multi-tenant enterprise platform (no teams, no shared workspaces)
 - Cursor or IDE replacement
 - Fully polished connector hub (GitHub is best; others are API-level)
 - 80%+ test-coverage certified product (currently 71%)
- Honest maturity: Strong personal/small-team RAG chat product with many adjacent features. Core loop works. Several features are backend-complete but frontend-thin.

2. Architecture

Browser (Next.js) -> FastAPI -> PostgreSQL + ChromaDB + Redis (optional) + Groq/OpenAI/Ollama
RQ Worker handles document ingestion and agent jobs.

Key code paths (verified)

- Chat: chat_routes.py -> tool_agent.py -> rag_service.py -> retrieval.py
- Upload: upload_routes.py -> upload_security_service.py -> ingestion_queue.py -> worker.py
- GitHub: github_connector_routes.py -> github_connector_service.py -> documents + RQ

Repository structure

- Backend: 24 API modules, 53 services, 35 DB tables, 16 migrations
- Frontend: 18 pages, 30 components, 8 hooks, 82 API client functions
- Tests: 53 backend test files (256 tests), 1 frontend test file (auth.test.ts)

3. Feature Inventory (Code-Verified)

3.1 Authentication

Feature	Backend	Frontend	Tests	Prod	Files
Google OAuth	Yes	Yes	Yes	Yes	oauth_routes.py
GitHub OAuth	Yes	Yes	Yes	Yes	oauth_routes.py
Cookie session + refresh	Yes	Yes	Yes	Yes	auth_service.py
CSRF protection	Yes	Yes	Partial	Yes	csrf.py
Logout	Yes	Yes	Yes	Yes	/auth/logout
Email/password register	Removed	Redirect	N/A	N/A	register/page.tsx
RBAC roles	Yes	Yes	Yes	Partial	rbac.py, /admin/rbac

3.2 Chat Platform

Feature	Backend	Frontend	Tests	Prod	
NDJSON streaming chat	Yes	Yes	Yes	Yes	/chat-stream
Sessions CRUD	Yes	Yes	Yes	Yes	session_routes.py
User memories	Yes	Yes	Yes	Yes	/memory
Chat folders + pin	Yes	Yes	Partial	Partial	no folder delete UI
Model picker	Yes	Yes	Yes	Yes	/models

Conversation summarization	Yes	Auto	Partial	Yes	memory_summary_service
----------------------------	-----	------	---------	-----	------------------------

3.3 RAG System

- Vector search (Chroma) - hybrid_search.py, rag/chroma_store.py - FULL
- Query rewriting - query_contextualizer_service.py - FULL
- Citations in responses - citation_service.py, sources-panel.tsx - FULL
- Session + collection scoped retrieval - FULL
- Reranking - reranker_service.py - PARTIAL (41% test coverage, optional)
- Redis retrieval cache - redis_cache_service.py - FULL if Redis configured

3.4 Document Intelligence

- Upload + quarantine scan - upload_routes.py, upload_security_service.py - FULL
- Background indexing via RQ worker - ingestion_queue.py, worker.py - FULL
- Paginated document list (40/page) - upload_routes.py, useDocuments.ts - FULL
- Indexing summary API - GET /documents/indexing-summary - FULL
- Insights: summary, FAQs, action items - document_intelligence_service.py - FULL
- Timeline + entity extraction - document_timeline, document_entities tables - FULL
- Auto-generate after index - ENABLE_DOCUMENT_INTELLIGENCE=true - FULL

3.5 Knowledge Graph

- graph_nodes, graph_edges tables - knowledge_graph_service.py - FULL
- Build, search, GraphRAG in chat - graph_routes.py, ENABLE_GRAPH_RAG - FULL
- UI: list/grid in workspace Graph tab + /knowledge-graph page - PARTIAL (no force-directed viz)
- Neo4j optional sync if NEO4J_URI set - OPTIONAL

3.6 Agents & Research

- Research agent POST /agents/research - wired to /research page - FULL
- Deep research pipeline POST /research/run - backend only, FE unused - PARTIAL
- Multi-agent orchestrator in chat (multi-agent mode) - FULL backend, trace list UI only
- Autonomous agents CRUD + manual run - /agents page - PARTIAL (scheduling needs Redis)
- Marketplace: 9 templates, install flow - /marketplace - FULL
- Research Markdown/PDF export APIs - no frontend buttons - PARTIAL

3.7 Connectors

Connector	OAuth	Sync	UI	Status
GitHub	Yes (dedicated)	Yes tarball+SHA	Chat GitHub tab	FULL
Notion	Token	Yes API	Token form /connectors	PARTIAL
Confluence	Token	Yes API	Hub card only	PARTIAL
Google Drive	No hub OAuth	Yes if token	Hub card only	PARTIAL
Dropbox	Token	Yes API	Hub card only	PARTIAL
Slack	No	Stub only	Listed only	SCAFFOLD

GitHub: manual sync only. No push webhooks. Logout does not disconnect GitHub.

3.8 Security

- Rate limiting (Redis + in-memory fallback) - production.py - FULL
- Upload validation, quarantine, MIME/ZIP checks - upload_security_service.py - FULL
- ClamAV integration - optional, CLAMAV_ENABLED=false default - PARTIAL
- Abuse / prompt injection detection - abuse_detection_service.py - FULL
- Audit logs + CSV export - audit_routes.py, /admin/audit - FULL
- Encrypted connector credentials (Fernet) - credential_crypto.py - FULL
- RBAC - DB exists; default gate is admin email allowlist - PARTIAL

3.9 Admin & Analytics

- Per-user analytics - GET /analytics/overview, /dashboard - FULL
- Platform analytics - admin only (403 for others) - FULL
- Audit center - /admin/audit - FULL

- RBAC assignment - /admin/rbac - FULL
- Ingestion queue admin API - no UI - PARTIAL

4. Database (35 tables, 16 migrations)

Migration head: 20260608_0016 (phases M-P: agents, connectors hub, marketplace)

- Core: users, chat_sessions, messages, documents, document_collections, user_memories
- Analytics: api_usage, model_usage, token_usage
- Intelligence: document_insights, document_timeline, document_entities
- Graph: graph_nodes, graph_edges | Agents: agent_traces, autonomous_agents, agent_executions
- GitHub: github_connections, github_repository_syncs
- Hub: connector_connections, connector_sync_runs
- Marketplace: marketplace_templates, marketplace_installs
- Settings: user_preferences, user_sessions, billing_*, security_audit_logs, user_roles
- Missing: teams/workspaces table (single-user scope only)
- Chroma vectors stored outside PostgreSQL in chroma_db/ path

5. API Surface (~125 endpoints)

Auth: /auth/* (OAuth, session, refresh, logout)

Chat: POST /chat-stream (primary), POST /chat (legacy)

Documents: /upload, /documents, /collections, /documents/indexing-summary

Intelligence: /documents/{id}/insights

Graph: /graph/build, /graph/search, /graph/global

Agents: /agents/research, /agents/multi-agent, /agents/traces, /agents/workspace/*

Research: /research/run, /research/reports (FE uses /agents/research instead)

GitHub: /connectors/github/* (real sync)

Hub: /connectors/hub/* (Notion, Confluence, Drive, Dropbox)

Admin: /audit/*, /admin/connectors/* (stub sync - do not use)

Backend-only (no FE): /notifications, /evaluation/run, /admin/ingestion-queue

6. Frontend Pages

- /chat - primary workspace (FULL)
- /login - OAuth only (FULL)
- /dashboard - analytics + quick actions (PARTIAL admin visibility)
- /settings - profile, 2FA, billing UI (PARTIAL for OAuth users)
- /agents, /research, /marketplace, /connectors, /knowledge-graph (LIVE)
- /admin/audit, /admin/rbac (admin-gated on API)
- /register, /forgot-password - redirect stubs to /login
- Landing page still links to /register (stale)
- Frontend tests: auth.test.ts only (5 tests). No E2E.

7. AI Capabilities

LLM providers in llm.py: Groq (default), OpenAI, Ollama. DeepSeek via ModelRouter.

NOT supported: Cursor API, Anthropic direct, Gemini direct.

LangGraph: NOT implemented. Custom Python orchestrator used.

- Chat + RAG: FULL
- Document intelligence: FULL
- Knowledge graph + GraphRAG: FULL (basic UI)
- Multi-agent chat: FULL backend, partial UI
- Autonomous scheduled agents: PARTIAL (needs Redis + RQ scheduler)
- Deep research dual APIs: PARTIAL (FE uses one path only)

8. Testing & Quality

Backend: 256 tests pass. Coverage 71.18% (pytest gate 80% FAILS).

Weak coverage: notification_service (29%), multi_document_service (44%), reranker (41%)

Strong coverage: upload_security, knowledge_graph, model_router, redis_cache

Frontend: 5 Vitest tests (auth helpers only). No Playwright E2E.

Untested E2E: connector hub Drive OAuth, Slack, notification UI, scheduled agent cron

9. Production Readiness (scores 0-100)

Area	Score	Notes
Core chat + RAG	85	Main loop complete and tested
Auth + security	80	OAuth, CSRF, rate limits, upload securit
Connectors	65	GitHub strong; others thin
Agents + research	70	Backend solid; UI gaps
Testing	55	71% coverage, no E2E
Multi-tenant enterprise	25	No teams/ACLs
Overall weighted	72	Good for self-hosted single-user/small t

Deploy requirements (from code)

- alembic upgrade head
- GROQ_API_KEY (or OPENAI + LLM_PROVIDER=openai)
- PostgreSQL, OAuth keys, FRONTEND_URL, AUTH_COOKIE_ENABLED=true
- REDIS_URL + python -m app.worker (ingestion)
- EMBEDDING_PROVIDER=openai or huggingface in prod (not local)
- UPLOAD_STAGING_DIR on persistent volume

10. Gap Analysis

FULLY IMPLEMENTED (end-to-end)

OAuth login, streaming RAG chat, sessions, memories, document upload/indexing, collections, document intelligence, knowledge graph, workspace search, GitHub connector, multi-agent chat, autonomous agents manual run, marketplace install, audit/RBAC (admin), rate limiting, model routing.

PARTIALLY IMPLEMENTED

- Dashboard: admin links visible to all users
- Deep research: two APIs; no export UI
- Connector hub: Notion token UI; others API-only
- Autonomous scheduling: RQ enqueue_at; poll_due_agents unused
- Notifications: API only, no bell UI
- Settings password/billing: UI exists; OAuth-only in practice
- GitHub: manual sync; no auto webhook on push

SCAFFOLD ONLY

- Slack connector (workspace_connector_service stub)
- Admin /admin/connectors/{id}/sync (returns stub message)
- Register/forgot-password pages (redirect)
- Billing (no payment processor)

NOT IMPLEMENTED

- Team workspaces / shared ACLs
- LangGraph
- Force-directed graph visualization
- Playwright E2E tests
- Eval results UI
- SSO/SAML
- GitHub push webhooks for auto-sync

- Cursor API as LLM provider

Documented but outdated

- docs/connectors.md says Notion/Confluence are stubs - backend has real sync code
- Phase docs claim 90%+ coverage - actual 71%

11. Bottom Line

Omni-AI Mark II is a real, working RAG chat application. The core loop (sign in -> upload or sync GitHub -> chat with citations) is implemented and tested. Document intelligence, knowledge graph, and GitHub connector are solid. Everything else ranges from backend-complete with thin UI to scaffold-only.

This report reflects the true state of the codebase as of 2026-05-25. Verify against code, not marketing copy or phase roadmaps.